



**MACAU UNIVERSITY OF SCIENCE AND
TECHNOLOGY**

School of Computer Science and Engineering

Faculty of Innovation Engineering

<<Software Project for Course Software Engineering>>

Homework ID : Task1-Project Proposal

Report Title : Instant Messaging Platform Across Network
Environments (Proposal)

Student Name : 游翔宇, 孙宇轩, 邢天舒

Student No. : 1230006152, 1230019445, 1230002381

Date : 9.27

Abstract

This proposal outlines a lightweight instant messaging platform designed for cross-network environments (LAN / public Internet). Building on the GitHub open-source project **socketio/chat-example**^[1], the system implements a real-time, bidirectional, event-driven chat room. It targets four major UX shortcomings in the prototype: a single channel only, difficult server deployment, missing user identity, and a rudimentary UI. Key improvements include: enabling one server to host multiple chat rooms, one-click deployment and room creation, user nicknames with an online user list, and front-end UI enhancements. The platform uses **Node.js** + **Socket.IO** as the foundation for real-time communication. The document presents a four-week implementation plan, success metrics, risks and mitigations, and an initial division of work.

Group number: 6

Group name: Do It Dui

References/URL:[socketio/chat-example: Basic chat example with Socket.IO](https://github.com/socketio/chat-example)

Table of Contents

Abstract	II
Table of Contents	III
Chapter 1 Introduction	1
1.1 Background and Motication	1
1.2 Team Profile	2
Chapter 2 Problem Diagnosis and Proposed Treatment	3
2.1 Problem Diagnosis	3
2.2 Proposed Treatment	5
Chapter 3 Work Plan & Key Technologies	6
3.1 Work Plan	6
3.1.1 MVP Setup	6
3.1.2 Database and Message History	6
3.1.3 UI Optimization and User Identity Management	6
3.1.4 Core Feature Expansion	6
3.2 Key Technologies	7
3.3 Success Metrics	8
References	9

Chapter 1 Introduction

1.1 Background and Motivation

With the proliferation of mobile Internet and instant messaging (IM) tools, the demand for communication that is “instant, convenient, and private” continues to grow. Mainstream platforms such as WeChat offer rich IM features, but they typically require user registration, client installation, or reliance on cloud services. In many ad-hoc scenarios, these prerequisites create excessive friction and fail to meet the expectation of “open and use immediately.” Users increasingly prefer new modes of communication that require no registration or installation, do not expose personal data, and are quick and convenient.

Moreover, in offline or constrained network settings (e.g., local area networks, hotspot peer-to-peer), there remains a need for convenient, localized communication methods—needs that mainstream tools often cannot satisfy.

Against this backdrop, this project aims to evolve a minimal Socket.IO-based chat prototype into a lightweight, self-hostable instant messaging platform that supports cross-network environments (LAN / public Internet).

Its core model is:

1. **Anyone can be a server:** Users can start a temporary chat room locally with a single click.
2. **Cross-network support:** Rooms are accessible via local LAN links and can optionally be exposed to the public Internet via NAT traversal.
3. **Zero-friction joining:** Other users need no registration or app installation; they can join directly in the browser via a link.

This model effectively addresses communication needs in ad-hoc, privacy-sensitive, and cross-network contexts, and provides a practical foundation for exploring decentralized instant messaging.

1.2 Team Profile

1. **The leader:** 游翔宇

Has experience in software project development and management, web development, and debugging multimodal large models based on the Transformer architecture. Responsible for requirements analysis, system architecture, front-end development, back-end development, project management, and schedule control.

2. **Group member:** 孙宇轩

Has experience in software engineering development and large model function development. Responsible for project back-end development, database management, front-end/back-end coordination and testing.

3. **Group member:** 邢天舒

Has experience in software project development and management. Responsible for front-end development, back-end development, testing and quality assurance, operations and deployment, and maintaining project documentation.

Chapter 2 Problem Diagnosis and Proposed Treatment

2.1 Problem Diagnosis

Although mainstream instant messaging (IM) software is feature-rich, it still falls short in specific scenarios such as ad-hoc communication, anonymity, and LAN-based exchange.

1. Establishing temporary communication with strangers often requires adding contacts, which is cumbersome and exposes personal information. Allowing users to join a public chat room via a temporary link would significantly reduce communication costs and protect social boundaries.
2. In events like lectures or salons with larger crowds, creating a group chat on short notice is impractical, whereas link-based joining enables rapid group interaction.
3. Cross-platform communication has high barriers: users on different devices, in different countries, and using different apps often cannot join the same group chat simultaneously. If connecting only requires a link and a browser, accessibility improves dramatically.
4. Moreover, most existing software depends entirely on the public Internet. In LAN environments such as meeting rooms or campus networks, local communication already suffices, with lower latency and less network overhead.
5. Finally, centralized architectures route information through third-party servers, lacking true privacy. If personal devices act directly as servers and distribute join links, we can achieve decentralization and stronger security.

Based on analysis of real-world scenarios and our understanding of the prototype *chat-example*, we summarize the core problems as follows:

1. **Centralized dependence and participation barriers:**
Current platforms rely heavily on cloud services and account systems; users must register and install dedicated clients to participate. This is too cumbersome for ad-hoc scenarios and hinders instant communication.

2. Lack of offline/LAN support:

Mainstream tools depend on public Internet servers and cannot operate properly in LAN or offline environments—a major limitation for workshops, temporary meetings, or offline settings.

3. Prototype limitations:

While the Socket.IO^[2]-based prototype validates real-time message broadcasting, it has the following shortcomings:

- (1) **No message history:** Refreshing the page or reconnecting loses all conversation content.
- (2) **Anonymous users:** Speakers cannot be distinguished; there is no identity or sense of how many are online.
- (3) **Spartan UI:** Poor interaction experience.

2.2 Proposed Treatment

The current *chat-example* demo has validated Socket.IO's real-time capability. Building on the issues above, we propose the following improvements:

1. **One-click room creation and cross-network access:**
 - (1) Server-side users can quickly start a temporary chat room and generate an access link.
 - (2) Two access modes:
 - ① Local direct connection in LAN/hotspot environments.
 - ② Public Internet access via NAT traversal.
 - (3) Browser-only joining: no registration, no installation.
2. **Message history storage and replay:**
 - (1) Chat content is persisted in a server-side database.
 - (2) New users automatically load a recent window of messages to preserve context.
3. **User identity and online list:**
 - (1) Users can set a nickname upon entry.
 - (2) The front end displays the online user list in real time.
4. **UI refinement and interaction improvements:**
 - (1) Provide an intuitive, clean, and aesthetically pleasing UI to reduce learning costs.
 - (2) Add basic emoji support to enhance the user experience.

Chapter 3 Work Plan & Key Technologies

3.1 Work Plan

To achieve the project goals, we divide development into four stages, roughly one week each.

3.1.1 MVP Setup

- Implement the basic chat page
- Enable local network connectivity and NAT traversal
- Provide automation scripts to generate access links

3.1.2 Database and Message History

- Select and design the database schema
- Implement message persistence
- Auto-load recent messages when a new user joins

3.1.3 UI Optimization and User Identity Management

- Design and polish the front-end UI
- Allow users to set a nickname when entering a room
- Implement real-time display of the online user list

3.1.4 Core Feature Expansion

- Perform system integration and end-to-end testing
- Fix bugs and optimize performance
- Optional features: message search, emoji sending

3.2 Key Technologies

1. Backend

- (1) **Node.js + Express.js:** Backend service framework for handling HTTP requests and middleware; hosts the Socket.IO server (JavaScript library).

2. Real-time Communication

- (1) **Socket.IO:** Event-based, real-time bidirectional communication supporting message broadcasting and user state management.
- (2) **Ngrok^[3]:** Tools for achieving internal network penetration

3. Database

- (1) **SQLite^[4]:** Lightweight embedded database for quick deployment and durable storage.

4. Frontend

- (1) **HTML + CSS + JavaScript:** Implements core UI and interaction logic.

3.3 Success Metrics

Whether the project meets expectations will be quantified by the following indicators:

1. Users can join a chat room directly.
2. Upon entry, users can load the 20 most recent messages.
3. Users can successfully set a nickname, which appears in the online user list.
4. New users can learn to send a message within 1 minute, with no training required.
5. The system stably supports at least 3 concurrent users in real-time conversation.
6. With guidance, a user can learn to create a chat room within 1 minute.

References

- [1] From the GitHub repository: [socketio/chat-example: Basic chat example with Socket.IO](#)
- [2] The socket official website: [Socket.IO](#)
- [3] The tool official website: [ngrok | API Gateway, Kubernetes Ingress, Webhook Gateway](#)
- [4] The database official website: [SQLite Home Page](#)